I. Finding Time Complexity of Algorithms

Problem 1: Finding Complexity using Counter Method

```
#include <stdio.h>

void function(int n)
{
   int i = 1;
   int s = 1;
   int count = 2;

   while (s <= n)
   {
      count++;
      i++;
      count++;

      s += i;
      count++;
   }

   count++;

   printf("%d", count);
}

int main()
{
   int n;
   scanf("%d", &n);
   function(n);
   return 0;
}
```

Problem 2: Finding Complexity using Counter method

```
#include <stdio.h>
void func(int n)
{
   int count = 0;
```

```
    if(n == 1)
    {
    }
    else
    {
        count++;
        for(int i = 1; i <= n; i++)
        {
            count++;

            count++;
            for(int j = 1; j <= n; j++)
            {
                count++;
                count++;
                break;
            }
            count++;
        }
        count++;
    printf("%d", count);
    }
}
int main()
{
    int n;
    scanf("%d", &n);
    func(n);
    return 0;
}
```

Problem 3: Finding Complexity using Counter Method

```
#include <stdio.h>
int main()
{
    int num, i, counter = 0;
    scanf("%d", &num);
    counter++;
    for (i = 1; i <= num; ++i)
    {
        counter++;
        if (num % i == 0)
        {
            counter++;
        }
```

```
        counter++;
    }
    printf("%d", counter);
    return 0;
}
```

Problem 4: Finding Complexity using Counter Method

```
#include <stdio.h>
void function(int n)
{
    int c = 0;
    c++;
    for (int i = n / 2; i < n; i++)
    {
        c++;
        for (int j = 1; j < n; j = 2 * j)
        {
            c++;
            for (int k = 1; k < n; k = k * 2)
            {
                c++;
                c++;
            }
            c++;
        }
        c++;
    }
    c++;
    printf("%d", c);
}
int main()
{
    int n;
    scanf("%d", &n);
    function(n);
    return 0;
}
```

Problem 5: Finding Complexity using Counter Method

```
#include <stdio.h>

int main()
```

```
{
   int n;
   int count = 3;

   scanf("%d", &n);

   while (n != 0)
   {
      count += 4;
      n = n / 10;
   }

   printf("%d", count);

   return 0;
}
```

II. Greedy Algorithms

Problem 1: Coin Problem

```
#include <stdio.h>

int main() {
   int V;
   scanf("%d", &V);

   int denominations[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
   int count = 0;

   for (int i = 0; i < 9; i++) {
      count += V / denominations[i];
      V %= denominations[i];
   }

   printf("%d", count);

   return 0;
}
```

Problem 2: Cookies Problem

```
#include <stdio.h>
#include <stdlib.h>
```

```
int compare(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;

    if (x < y)
        return -1;
    if (x > y)
        return 1;
    return 0;
}

int main()
{
    int n, m;

    scanf("%d", &n);

    int g[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &g[i]);

    scanf("%d", &m);

    int s[m];
    for (int i = 0; i < m; i++)
        scanf("%d", &s[i]);

    qsort(g, n, sizeof(int), compare);
    qsort(s, m, sizeof(int), compare);

    int child = 0;
    int cookie = 0;
    int content = 0;

    while (child < n && cookie < m)
    {
        if (s[cookie] >= g[child])
        {
            content++;
            child++;
            cookie++;
        }
```

```
        else
        {
            cookie++;
        }
    }

    printf("%d", content);

    return 0;
}
```

Problem 3: Burger Problem

```
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b)
{
    long long x = *(const long long *)a;
    long long y = *(const long long *)b;

    if (x < y)
        return 1;
    if (x > y)
        return -1;
    return 0;
}

void printInt128(__int128 n)
{
    if (n == 0)
    {
        printf("0");
        return;
    }

    if (n < 0)
    {
        putchar('-');
        n = -n;
    }

    char digits[10000];
    int i = 0;
```

```
    while (n > 0)
    {
        digits[i++] = '0' + (n % 10);
        n /= 10;
    }

    while (i--)
        putchar(digits[i]);
}

int main()
{
    int n;
    scanf("%d", &n);

    long long calories[n];

    for (int i = 0; i < n; i++)
        scanf("%lld", &calories[i]);

    /* Highest calorie burger must be eaten first */
    qsort(calories, n, sizeof(long long), compare);

    __int128 distance = 0;
    __int128 power = 1;

    for (int i = 0; i < n; i++)
    {
        distance += (__int128)calories[i] * power;
        power *= 3;
    }

    printInt128(distance);

    return 0;
}
```

Problem 4: Array Sum Max Problem

```
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b)
```

```
{
    long long x = *(const long long *)a;
    long long y = *(const long long *)b;

    if (x < y)
        return -1;
    if (x > y)
        return 1;
    return 0;
}

int main()
{
    int n;
    scanf("%d", &n);

    long long arr[n];

    for (int i = 0; i < n; i++)
        scanf("%lld", &arr[i]);

    // Sort in ascending order
    qsort(arr, n, sizeof(long long), compare);

    long long sum = 0;

    for (int i = 0; i < n; i++)
    {
        sum += arr[i] * i;
    }

    printf("%lld", sum);

    return 0;
}
```

Problem 5: Product of Array Elements - Minimum

```
#include <stdio.h>
#include <stdlib.h>

int ascending(const void *a, const void *b)
{
    int x = *(const int *)a;
```

```
    int y = *(const int *)b;

    return (x > y) - (x < y);
}

int descending(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;

    return (y > x) - (y < x);
}

int main()
{
    int n;
    scanf("%d", &n);

    long long A[n], B[n];

    for (int i = 0; i < n; i++)
        scanf("%lld", &A[i]);

    for (int i = 0; i < n; i++)
        scanf("%lld", &B[i]);

    qsort(A, n, sizeof(long long), ascending);
    qsort(B, n, sizeof(long long), descending);

    long long sum = 0;

    for (int i = 0; i < n; i++)
        sum += A[i] * B[i];

    printf("%lld", sum);

    return 0;
}
```

III. Divide & Conquer

Problem 1: Number of Zeroes in a Given Array

```
#include <stdio.h>
```

```
int findFirstZero(int arr[], int low, int high)
{
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == 0)
    {
        // Check if this is the first zero
        if (mid == 0 || arr[mid - 1] == 1)
            return mid;

        return findFirstZero(arr, low, mid - 1);
    }
    else
    {
        return findFirstZero(arr, mid + 1, high);
    }
}

int main()
{
    int m;
    scanf("%d", &m);

    int arr[m];

    for (int i = 0; i < m; i++)
        scanf("%d", &arr[i]);

    int firstZero = findFirstZero(arr, 0, m - 1);

    if (firstZero == -1)
        printf("0");
    else
        printf("%d", m - firstZero);

    return 0;
}
```

Problem 2: Majority Element

```
#include <stdio.h>
```

```
int majorityElement(int* nums, int size) {
    int candidate = nums[0];
    int count = 1;
    for (int i = 1; i < size; i++) {
        if (nums[i] == candidate) {
            count++;
        } else {
            count--;
            if (count == 0) {
                candidate = nums[i];
                count = 1;
            }
        }
    }
    return candidate;
}
int main() {
    int n;
    scanf("%d", &n);
    int nums[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }
    int result = majorityElement(nums, n);
    printf("%d\n", result);
    return 0;
}
```

Problem 3: Finding Floor Value

```
#include <stdio.h>

int findFloor(int arr[], int low, int high, int x)
{
    if (low > high)
        return -1;

    int mid = (low + high) / 2;

    if (arr[mid] == x)
        return arr[mid];

    if (arr[mid] < x)
    {
```

```
        int result = findFloor(arr, mid + 1, high, x);

        if (result == -1)
            return arr[mid];

        return result;
    }

    return findFloor(arr, low, mid - 1, x);
}

int main()
{
    int n, x;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &x);

    printf("%d", findFloor(arr, 0, n - 1, x));

    return 0;
}
```

Problem 4: Two Elements Sum to x

```
#include <stdio.h>

int binarySearch(int arr[], int low, int high, int target)
{
    if (low > high)
        return -1;

    int mid = (low + high) / 2;

    if (arr[mid] == target)
        return mid;

    if (arr[mid] < target)
```

```
        return binarySearch(arr, mid + 1, high, target);

    return binarySearch(arr, low, mid - 1, target);
}

int findPair(int arr[], int n, int x, int i, int *a, int *b)
{
    if (i >= n)
        return 0;

    int target = x - arr[i];

    int j = binarySearch(arr, i + 1, n - 1, target);

    if (j != -1)
    {
        *a = arr[i];
        *b = arr[j];
        return 1;
    }

    return findPair(arr, n, x, i + 1, a, b);
}

int main()
{
    int n, x, a, b;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &x);

    if (findPair(arr, n, x, 0, &a, &b))
    {
        printf("%d\n", a);
        printf("%d\n", b);
    }
    else
    {
```

```
        printf("No");
    }

    return 0;
}
```

Problem 5: Implementation of Quick Sort

```
#include <stdio.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);

    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
```

```
    }
}

int main()
{
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    quickSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

IV. Dynamic Programming

Problem 1: Playing with Numbers

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    long long dp[n + 1];

    for (int i = 0; i <= n; i++)
        dp[i] = 0;

    dp[0] = 1;

    for (int i = 1; i <= n; i++)
    {
        dp[i] = dp[i - 1];

        if (i >= 3)
```

```
        dp[i] += dp[i - 3];
    }

    printf("%lld", dp[n]);

    return 0;
}
```

Problem 2: Playing with Chessboard

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    long long board[n][n];
    long long dp[n][n];

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%lld", &board[i][j]);
        }
    }

    dp[0][0] = board[0][0];

    // First row
    for (int j = 1; j < n; j++)
    {
        dp[0][j] = dp[0][j - 1] + board[0][j];
    }

    // First column
    for (int i = 1; i < n; i++)
    {
        dp[i][0] = dp[i - 1][0] + board[i][0];
    }

    // Remaining cells
    for (int i = 1; i < n; i++)
```

```
    {
        for (int j = 1; j < n; j++)
        {
            if (dp[i - 1][j] > dp[i][j - 1])
                dp[i][j] = dp[i - 1][j] + board[i][j];
            else
                dp[i][j] = dp[i][j - 1] + board[i][j];
        }
    }

    printf("%lld", dp[n - 1][n - 1]);

    return 0;
}
```

Problem 3: Longest Common Subsequence

```
#include <stdio.h>
#include <string.h>

int main()
{
    char s1[1001], s2[1001];

    scanf("%s", s1);
    scanf("%s", s2);

    int n = strlen(s1);
    int m = strlen(s2);

    int dp[n + 1][m + 1];

    // Initialize first row and first column
    for (int i = 0; i <= n; i++)
        dp[i][0] = 0;

    for (int j = 0; j <= m; j++)
        dp[0][j] = 0;

    // Build the DP table
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
```

```
        {
            if (s1[i - 1] == s2[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
            {
                if (dp[i - 1][j] > dp[i][j - 1])
                    dp[i][j] = dp[i - 1][j];
                else
                    dp[i][j] = dp[i][j - 1];
            }
        }
    }

    printf("%d", dp[n][m]);

    return 0;
}
```

Problem 4: Longest Non-decreasing Subsequence

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int arr[n];
    int dp[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
        dp[i] = 1;
    }

    int maxLength = 1;

    for (int i = 1; i < n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (arr[j] <= arr[i] && dp[j] + 1 > dp[i])
            {
```

```
            dp[i] = dp[j] + 1;
        }
    }

    if (dp[i] > maxLength)
        maxLength = dp[i];
}

printf("%d", maxLength);

return 0;
}
```

V. Competitive Programming

Problem 1: Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n;
    scanf("%d", &n);

    int *seen = calloc(n + 1, sizeof(int));
    int x;

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &x);

        if (seen[x] == 1)
        {
            printf("%d", x);
            free(seen);
            return 0;
        }

        seen[x] = 1;
    }

    free(seen);
```

```
    return 0;
}
```

Problem 2: Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int* count = (int*)calloc(n + 1, sizeof(int));
    int result = -1;

    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        count[val]++;
        if (count[val] == 2) {
            result = val;
        }
    }

    printf("%d\n", result);

    free(count);
    return 0;
}
```

Problem 3: Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity

```
#include <stdio.h>
#include <stdlib.h>

void solve() {
    int n1;
    if (scanf("%d", &n1) != 1) return;
    int* a = (int*)malloc(n1 * sizeof(int));
    for (int i = 0; i < n1; i++) {
        scanf("%d", &a[i]);
    }
```

```
    int n2;
    scanf("%d", &n2);
    int* b = (int*)malloc(n2 * sizeof(int));
    for (int i = 0; i < n2; i++) {
        scanf("%d", &b[i]);
    }

    int i = 0, j = 0;
    int first = 1; // Used for formatting spaces between outputs

    while (i < n1 && j < n2) {
        if (a[i] == b[j]) {
            if (!first) printf(" ");
            printf("%d", a[i]);
            first = 0;
            i++;
            j++;
        } else if (a[i] < b[j]) {
            i++;
        } else {
            j++;
        }
    }
    printf("\n");

    free(a);
    free(b);
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        solve();
    }
    return 0;
}
```

Problem 4: Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity

```
#include <stdio.h>

void solve() {
```

```
    int n1, n2;

    // Read first array
    if (scanf("%d", &n1) != 1) return;
    int a[n1];
    for (int i = 0; i < n1; i++) {
        scanf("%d", &a[i]);
    }

    // Read second array
    scanf("%d", &n2);
    int b[n2];
    for (int i = 0; i < n2; i++) {
        scanf("%d", &b[i]);
    }

    // Two-pointer approach
    int i = 0, j = 0;
    int first = 1;

    while (i < n1 && j < n2) {
        if (a[i] == b[j]) {
            if (!first) printf(" ");
            printf("%d", a[i]);
            first = 0;
            i++;
            j++;
        } else if (a[i] < b[j]) {
            i++;
        } else {
            j++;
        }
    }
    printf("\n");
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        solve();
    }
    return 0;
}
```

Problem 5: Pair with Difference-O(n^2) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int* A = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }

    int k;
    scanf("%d", &k);

    int i = 0, j = 1;
    int found = 0;

    while (i < n && j < n) {
        if (i == j) {
            j++;
            continue;
        }

        int diff = A[j] - A[i];

        if (diff == k) {
            found = 1;
            break;
        } else if (diff < k) {
            j++;
        } else {
            i++;
        }
    }

    printf("%d\n", found);

    free(A);
    return 0;
```

}

Problem 6: Pair with Difference-O(n) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int* A = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }

    int k;
    scanf("%d", &k);

    int i = 0, j = 1;
    int found = 0;

    while (i < n && j < n) {
        if (i == j) {
            j++;
            continue;
        }

        int diff = A[j] - A[i];

        if (diff == k) {
            found = 1;
            break;
        } else if (diff < k) {
            j++;
        } else {
            i++;
        }
    }

    printf("%d\n", found);

    free(A);
    return 0;
```

}